

Project Patent — Submission Portal Design Plan

1. Data Model

1.1 Core Entities

Submission

The central object. Everything hangs off a submission.

Field	Type	Notes
id	UUID	Primary key
reference_number	string	Human-readable ID (e.g., SUB-2026-00142)
status	enum	received , open , under_review , proposal_produced , bound , declined , closed
insured_name	string	Named insured from intake
effective_date	date	Requested policy effective
expiration_date	date	Requested policy expiration
line_of_business	enum	occurrence_gl , habitational , contractor , small_biz_casualty
premium_indication	decimal	Initial indicated premium (system-generated)
assigned_underwriter_id	FK → User	Who owns this submission
approver_id	FK → User	Authority/sign-off
source	string	Broker name or submission channel
broker_contact_email	string	For correspondence
created_at	timestamp	When submission entered the system

updated_at	timestamp	Last modification
------------	-----------	-------------------

Status flow:

```
received → open → under_review → proposal_produced → bound
                                     → declined
                                     → closed (at any point)
```

Proposal

N proposals per submission. Each represents a distinct quote option.

Field	Type	Notes
id	UUID	Primary key
submission_id	FK → Submission	Parent submission
version	integer	Auto-incrementing per submission (v1, v2, v3...)
label	string	Optional name (e.g., "Good", "Better", "Best")
status	enum	draft , pending_review , approved , issued , declined
total_premium	decimal	Calculated total
commission_rate	decimal	Broker commission %
created_by_id	FK → User	Who generated this version
created_at	timestamp	
snapshot	JSONB	Frozen copy of all rates, forms, debits/credits at time of creation — enables version diffing

Key design decision: The `snapshot` field stores a serialized copy of the proposal's full state at creation time. This is what powers the version delta comparison feature — you diff `snapshot` between v1 and v2 rather than trying to reconstruct history from mutable child records.

Form

Insurance forms attached to a proposal.

Field	Type	Notes
id	UUID	Primary key
proposal_id	FK → Proposal	Parent proposal
form_number	string	ISO/proprietary form ID (e.g., CG 00 01 04 13)
form_name	string	Human-readable title
form_edition_date	string	Edition date of the form
form_type	enum	coverage , endorsement , exclusion , condition
is_mandatory	boolean	Required by underwriting rules
sort_order	integer	Display ordering

DebitCredit

Premium modifications tied to a form within a proposal. Combined with Form on the UI.

Field	Type	Notes
id	UUID	Primary key
proposal_id	FK → Proposal	Parent proposal
form_id	FK → Form	Which form this adjustment relates to (nullable for submission-level adjustments)
adjustment_type	enum	debit , credit
description	string	What the adjustment is for
percentage	decimal	Percentage modification (e.g., +15%, -10%)
flat_amount	decimal	Alternative: flat dollar adjustment
is_default	boolean	Whether this is the system-default for this form
applied_by_id	FK → User	Who made the adjustment

Design note: Each form has a set of default debits/credits populated by the rating engine. Underwriters can override or add custom adjustments. The `is_default` flag distinguishes system-generated from manual.

Rate

Rating data populated by the rating engine (Satora output).

Field	Type	Notes
id	UUID	Primary key
submission_id	FK → Submission	Parent submission
classification_code	string	Class code
classification_description	string	
territory	string	Rating territory
base_rate	decimal	Base rate per \$1K
exposure	decimal	Revenue, payroll, area, units — depends on basis
exposure_basis	enum	<code>revenue , payroll , area , units , other</code>
indicated_premium	decimal	Calculated premium for this line
loss_cost	decimal	If loss-cost rated
effective_date	date	Rate as-of date
source	string	Rating engine or manual entry

LossRun

Loss history data extracted from submission documents.

Field	Type	Notes
id	UUID	Primary key

submission_id	FK → Submission	
policy_year	integer	
carrier	string	Prior carrier name
premium	decimal	Earned premium for that year
incurred_losses	decimal	Total incurred
paid_losses	decimal	Total paid
open_reserves	decimal	
claim_count	integer	Number of claims
large_loss_threshold	decimal	Definition used
large_losses	decimal	Losses above threshold
loss_ratio	decimal	Calculated
valuation_date	date	As-of date for the loss data

Document

Supporting documents uploaded with or attached to the submission.

Field	Type	Notes
id	UUID	Primary key
submission_id	FK → Submission	
filename	string	Original filename
file_type	enum	<code>acord_app</code> , <code>loss_runs</code> , <code>supplemental</code> , <code>photos</code> , <code>financials</code> , <code>other</code>
storage_path	string	S3 key or equivalent
uploaded_by_id	FK → User	
uploaded_at	timestamp	
		<code>pending</code> , <code>processing</code> , <code>completed</code> , <code>failed</code> —

extraction_status	enum	Satora extraction state
extracted_data	JSONB	Structured output from Satora

Note

Notes attached to a submission (human or system-generated).

Field	Type	Notes
id	UUID	Primary key
submission_id	FK → Submission	
author_id	FK → User	Nullable for system notes
content	text	Note body
note_type	enum	<code>manual</code> , <code>system</code> , <code>email</code>
is_internal	boolean	Internal vs. broker-visible
created_at	timestamp	

Email

Emails associated with a submission.

Field	Type	Notes
id	UUID	Primary key
submission_id	FK → Submission	
direction	enum	<code>inbound</code> , <code>outbound</code>
from_address	string	
to_addresses	string[]	
subject	string	
body_text	text	Plain text body
body_html	text	HTML body

`extracted_data` JSONB field that Satora populates asynchronously after upload.

3. **Forms + Debits/Credits as siblings** — Both hang off Proposal with a join between them. This supports the combined UI screen you described.
 4. **Email as first-class entity** — Not just attachments. Supports the forwarding workflow where emails can be sent to a submission inbox and auto-linked.
-

2. Screen Design Plan

Screen 1: Submission List

Purpose: Central dashboard. Used for triage, assignment, and navigation.

Layout: Filterable data table with bulk actions.

Columns:

- Reference number
- Insured name
- Status (color-coded badges)
- Line of business
- Indicated premium
- Proposal count (e.g., "2 proposals")
- Assigned underwriter
- Approver
- Received date
- Last updated

Interactions:

- Click row → navigates to Submission Detail
- Filter bar: status, LOB, underwriter, date range
- Bulk assign: select multiple → assign underwriter / approver via dropdown
- Sort by any column

- Search by insured name or reference number
- Status filter tabs across the top (All / Received / Open / Under Review / Proposal Produced / Closed)

Auto-refresh: New submissions appear in real-time or on short poll (received status, unassigned).

Screen 2: Submission Detail (Container)

Purpose: Single submission view. Tab-based or sidebar navigation to sub-screens.

Header area (persistent across tabs):

- Reference number, insured name, status badge
- Assigned underwriter + approver (editable inline)
- Effective/expiration dates
- Quick actions: change status, assign, create new proposal

Tabs / Sub-navigation:

Tab 2A: Overview

- Summary card: key fields from Satora extraction (insured info, operations description, requested limits, prior coverage summary)
- Status timeline showing progression
- Proposal cards: each proposal shown as a card with label, version, total premium, status. Click → opens Proposal Detail.
- Action: "Generate Proposal" button triggers auto-proposal creation

Tab 2B: Documents

- List of all uploaded documents with file type tags
- Upload button for manual additions
- Extraction status indicator per document (pending / processing / completed / failed)
- Click document → preview (PDF viewer or image viewer inline)
- Satora structured output expandable per document

Tab 2C: Rates

- Table showing all rate lines from the rating engine
- Classification code, description, territory, base rate, exposure, indicated premium
- Editable cells for underwriter overrides (highlighted when modified from system default)
- Total indicated premium at bottom
- Source attribution (which rating run produced these)

Tab 2D: Loss Runs

- Table by policy year: carrier, premium, incurred, paid, reserves, claim count, loss ratio
- Summary row: 3-year and 5-year weighted averages
- Large loss detail expandable
- Valuation date noted
- Upload additional loss runs

Tab 2E: Notes & Emails

- Combined chronological feed (notes + emails interleaved by date)
- Add note form (with internal/external toggle)
- Email display: from, to, subject, body (expandable)
- Forward email action: enter email address → system sends with submission context
- Filter: notes only, emails only, all

Tab 2F: Structured Data (Satora Output)

- Accordion or card layout showing all fields Satora extracted
- Grouped by document source
- Confidence indicators per field
- Editable — underwriter can correct/override extracted values
- “Re-extract” button if documents are updated

Screen 3: Proposal Detail

Purpose: View and edit a single proposal. Accessed from Submission Detail → Overview

tab.

Header:

- Proposal label + version (e.g., "Best — v2")
- Status badge
- Total premium
- Created by / created date
- Actions: approve, decline, create new version, export/print

Sections:

Section 3A: Forms & Debits/Credits (Combined Screen)

This is the combined view you described. Each form is a row or expandable card:

Form Number	Form Name	Type	Default Adj.	Custom Adj.	Net Impact
CG 00 01	Commercial GL Coverage	Coverage	—	—	Base
CG 21 06	Exclusion – Pollution	Exclusion	-5%	—	-5%
CG 24 04	Waiver of Subrogation	Endorsement	+10%	+12% (override)	+12%

- Each row expandable to show/edit individual debit/credit line items
- "Add Form" button with searchable form library
- "Add Adjustment" within any form row
- Running total premium updates in real-time as adjustments change
- Visual indicator when a default has been overridden

Section 3B: Premium Summary

- Base premium (from rates)
- Itemized adjustments (debits subtotal, credits subtotal)
- Expense constants, taxes, fees

- Total premium
- Commission amount

Section 3C: Version Comparison (Future — V2)

- Side-by-side or diff view between two proposal versions
 - Highlights: forms added/removed, adjustments changed, premium delta
 - Powered by diffing the `snapshot` JSONB between versions
-

3. Build Sequence

Phase 1: Foundation (Weeks 1–3)

1. **Data model migration** — Create all tables, indexes, constraints
2. **Submission List screen** — Table, filters, status management, assignment
3. **Submission Detail container** — Header, tab navigation, basic overview
4. **Document upload + storage** — S3 integration, file type detection

Phase 2: Intelligence Layer (Weeks 4–6)

5. **Satora integration** — Async extraction pipeline, structured data display
6. **Rate screen** — Rating engine integration, display, overrides
7. **Loss Run screen** — Data model population, summary calculations
8. **Auto-proposal generation** — System creates initial proposal from extracted data + rates

Phase 3: Proposal Workflow (Weeks 7–9)

9. **Proposal Detail screen** — Full proposal view with premium summary
10. **Forms & Debits/Credits combined screen** — Form library, adjustments, real-time totals
11. **Notes & Emails** — Note creation, email display, forwarding
12. **Proposal status workflow** — Draft → review → approval routing

Phase 4: Polish & Diff (Weeks 10–12)

13. **Version snapshotting** — Freeze proposal state on version creation
 14. **Version comparison** — Diff engine + UI for side-by-side comparison
 15. **Bulk operations** — Multi-select assignment, status changes
 16. **Audit trail** — Log all state changes, assignments, overrides
-

4. Open Design Questions

1. **Forms + Debits/Credits layout** — Inline editable table vs. expandable cards vs. side panel? Recommend: expandable rows in a table (compact view with drill-down).
2. **Version diff granularity** — Field-level diff (every changed value highlighted) or section-level summary (forms added/removed, premium changed)? Recommend: start with section-level, add field-level in a later pass.
3. **Email integration** — Inbound parsing (dedicated submission inbox address like `sub-00142@submissions.patent.com`) vs. manual forwarding only? Recommend: start with manual forward, build inbound parsing as V2.
4. **Authority routing** — Should proposal approval automatically route based on `authority_limit` on the User model, or is it manual? Recommend: auto-route with manual override.
5. **Real-time updates** — WebSocket for live submission list updates, or polling? Depends on expected volume. For <100 concurrent users, polling at 30s intervals is fine.
6. **Good-Better-Best packaging** — Should the system auto-generate all three tiers from a single submission, or does the underwriter manually create each? Recommend: auto-generate with manual override capability.